

LangChain vs LangGraph — Study Notes

Notes on why LangGraph exists, what problems it solves that LangChain cannot, and a side-by-side comparison of the two libraries.

1. What is LangChain?

LangChain is an open-source library designed to simplify the process of building LLM-based applications (i.e., applications that integrate large language models — chatbots, browser plugins that let you chat with content, RAG systems, etc.).

It provides several modular building blocks that can be combined to construct any kind of LLM-based workflow:

- **Model** — a unified interface to talk to any LLM provider (OpenAI, Anthropic, Hugging Face, Ollama, etc.) without major code changes when switching providers.
- **Prompts** — supports prompt engineering and templating.
- **Retrievers** — fetch relevant documents/context from a vector store or knowledge base, using different retrieval strategies and algorithms; the foundation of RAG applications.
- **Chains** — the biggest offering of LangChain. Chains let you connect components (e.g., prompt → model → output parser) so the output of one block automatically feeds the next, letting you build workflows of any length without manually wiring the handoff.

What you can build with LangChain:

- Simple conversational workflows — chatbots, text summarizers (prompt → LLM → display output, optionally looped).
- Multi-step workflows — e.g., topic → detailed report → summary, by chaining prompt/LLM/output-parser components.
- RAG-based applications — user prompt → retriever fetches relevant context → context + prompt sent to LLM → response.
- Basic tool-using agents — LLMs can be connected to tools (APIs or Python functions) and decide when to call which tool with what input (e.g., calling a weather API tool when asked about today's weather).

2. Why LangGraph? — Workflow vs. Agent

Before comparing LangChain and LangGraph, it's useful to distinguish a “workflow” from an “agent” (as described in Anthropic's “Building Effective Agents” article):

- **Workflows:** systems where LLMs and tools are orchestrated through predefined code paths. The flow chart is fixed and built by a developer; it executes in the same order every single time (run it once or a hundred times — same information flow).
- **Agents:** systems where LLMs dynamically direct their own processes and tool usage, maintaining control over how they accomplish a task. The flow chart is decided dynamically by the agent itself and can differ from run to run.

Both use LLMs, but one is more autonomous (agent) and one is created/fixed by humans (workflow). A complex automated process (e.g., an automated hiring pipeline: receive a hiring request → create a job description → get human approval → post the job → wait and monitor applications → modify and repeat if needed → shortlist candidates → schedule interviews → conduct interviews → send offer or regret letters → negotiate if needed → onboard) is a good example of a workflow: it has clear conditional branches, loops, and long waits — exactly the kind of structure that is easy to draw as a flow chart but hard to implement as a simple linear chain.

3. Eight Reasons LangGraph Exists (Challenges with LangChain)

When trying to implement a complex, non-linear workflow (like the hiring example above) purely in LangChain, several concrete challenges appear. LangGraph was built specifically to solve them.

1. Control Flow Complexity

LangChain is built primarily for chains, which are linear workflows: output of block 1 feeds block 2, and so on.

Complex workflows often need non-linear control flow due to three factors:

- Conditional branches — the flow goes one way or another based on a condition (e.g., “enough applications?” yes/no).
- Loops — repeating a step until a condition is met (e.g., keep regenerating a job description until approved).
- Jumps — control suddenly moves forward or backward in the logic (e.g., going back to an earlier step after a wait period).

LangChain's limitation: LangChain has no built-in constructs for conditional branching, loops, or jumps. Developers must write this logic manually in plain Python (while loops, if/else) outside the library. This extra code is called “glue code.” The more glue code a project accumulates, the harder it becomes to maintain, debug, and collaborate on.

How LangGraph solves it: LangGraph represents the entire workflow as a graph. Each task becomes a node, and edges (including conditional edges) define the control flow. Since a graph is inherently a non-linear data structure, branching, looping, and jumping are all native, first-class capabilities — no custom glue code required.

2. Handling State

In any non-trivial workflow, there are data points whose values evolve over time as execution progresses (e.g., the job description text, whether it's approved, how many applications have arrived, shortlisted candidates, offer status, onboarding status). Collectively, these evolving data points are called the “state” of the workflow. A workflow cannot function correctly without properly tracking its state.

LangChain's limitation: LangChain has no built-in mechanism to store and track this kind of key-value state across a workflow. It only offers conversational memory (storing past chat messages), not general application state. To implement state manually, developers must create and manage a plain dictionary themselves throughout the chain and update it by hand at every step — workflows built this way are effectively stateless from the framework's point of view, and manual state management is error-prone in complex flows.

How LangGraph solves it: LangGraph execution is stateful by design. When you create a graph, you also define a state object (e.g., using Pydantic or a TypedDict). This state is accessible (readable and mutable) by every node in the graph. Each node receives the current state as input, can read and update it, and passes the updated state to the next node automatically — no manual dictionary-passing is required, no matter how many fields the state has.

3. Event-Driven Execution

A workflow can execute in two ways: Sequential — runs start to finish without stopping; or Event-driven — execution pauses at some point and waits for an external trigger (e.g., a timer elapsing, a candidate's reply) before resuming.

LangChain's limitation: LangChain was designed for sequential execution — once a chain starts, it runs to completion. It has no native way to pause for days and resume later. To approximate this, developers must split the logic into separate chains, write external code to track elapsed time or wait for triggers, and manually transfer state between the chains — again resulting in significant glue code.

How LangGraph solves it: Because LangGraph execution is stateful, it can save the current state at any node using a mechanism called a checkpoint (in memory or in an external database), pause execution, and later resume exactly from where it left off once the external trigger occurs. Event-driven execution is built into LangGraph's design.

4. Fault Tolerance

Fault tolerance means a system's ability to recover and keep functioning correctly after something goes wrong. It matters most in long-running workflows, where the chance of a fault (small, e.g. an API being down; or big, e.g. the server/container crashing) is higher.

LangChain's limitation: LangChain has no concept of fault tolerance. If a 5-step chain fails at step 3, it must be re-executed from the very beginning, because LangChain assumes its chains are short-lived (trigger → run → finish).

How LangGraph solves it: LangGraph provides both retry logic (catch an error such as an API failure and retry after a delay) for small faults, and recovery for large/system-level faults. Because every node's execution creates a checkpoint (a snapshot of the state), if the whole system crashes, execution can be resumed exactly from the last successful node rather than restarting from scratch.

5. Human-in-the-Loop

Human-in-the-loop means pausing a workflow at a particular stage to get a decision or approval from a human (e.g., approving a job description before it's posted, or approving before posting anything publicly). This matters because accountability for risky actions should rest with a human, not the automated system.

LangChain's limitation: LangChain has no default mechanism for a chain to pause indefinitely, wait for a human, and resume later. You can ask for human input mid-chain, but only practically for short waits — because LangChain chains are synchronous and sequential. If human approval can take a long time (e.g., 24 hours), the script would sit idle consuming compute resources and risks crashing. A workaround is splitting the chain into two parts and manually passing state between them when resuming — again glue code and maintainability issues.

How LangGraph solves it: In LangGraph, human-in-the-loop is a first-class citizen, explicitly documented as a core capability. LangGraph can pause execution indefinitely — for minutes, hours, or even days — until human input is received, because it checkpoints the graph state after every step. This lets the system persist execution context and later resume the workflow continuing from where it left off, supporting asynchronous human review without time constraints (much like saving progress in a video game and resuming later from the same stage).

6. Nested Workflows (Subgraphs)

Not exactly a “challenge” for LangGraph so much as a feature it has that LangChain lacks.

LangChain's limitation: LangChain has no equivalent feature — there is no way to embed one workflow inside another as a reusable, composable unit.

A subgraph is a graph that is used as a node inside another graph — a concept of encapsulation applied to graphs. In other words, you can replace any single node with an entire nested graph.

- Building multi-agent systems — a system where multiple agents work together, each handling a different responsibility (example: a self-driving car system with separate agents for sensor processing, driving, entertainment, and a coordinating “CEO” agent).
- Reusability — building a small, reusable graph (e.g., an approval sub-workflow) once and reusing it at multiple points inside a larger graph, similar to reusing functions in programming.

7. Observability

Observability refers to how easily you can monitor, debug, and understand what a workflow is doing at run time. This matters a lot in production, both for debugging and for auditing (e.g., understanding why an agent made a costly or incorrect decision).

LangChain's limitation: LangChain integrates with LangSmith (a monitoring library) which records LLM calls, prompts sent, replies received, tokens used, and response time. However, LangSmith can only monitor LangChain's own code — it cannot see inside custom glue code (e.g., a manual while loop or manual state dictionary). So for any complex workflow built with LangChain, observability ends up only partial, not complete.

How LangGraph solves it: Because LangGraph execution is entirely stateful and everything (including control flow, loops, and human approvals) happens inside LangGraph itself rather than in external glue code, LangGraph

has a very tight integration with LangSmith. It tracks a full chronological timeline of every node's execution, state changes before/after each node, messages exchanged, and exactly when and where human approvals occurred — giving complete observability rather than partial.

4. LangGraph — Core Concepts

LangGraph represents any workflow as a graph:

- Node — represents a single task/step (a plain Python function, e.g., “create job description”, “check approval”, “post job description”).
- Edge — connects nodes and determines control flow. Conditional edges route execution to different nodes based on a condition.
- State object — a (Pydantic model or TypedDict) dictionary accessible to every node; nodes read from it and write updates to it. This is what makes execution stateful.
- Checkpointer — saves a snapshot of the state after every node executes (in memory or an external database); enables pausing, resuming, fault recovery, and long-running human-in-the-loop waits.
- Subgraph — a graph used as a node inside another graph, enabling nested workflows, multi-agent systems, and reusable sub-workflows.

Definition to remember: “LangGraph is an orchestration framework that enables building stateful, multi-step, and event-driven workflows using LLMs. It's ideal for designing both single-agent and multi-agent AI applications. Think of it as a flow-chart engine for LLMs — you define steps as nodes, how they're connected using edges, and the logic that governs the transition. LangGraph takes care of state management, conditional branching, looping, pausing and resuming, and fault recovery — features essential for building robust, production-grade AI systems.”

5. LangChain vs LangGraph — Side by Side

Aspect	Description
Full name	Open-source library designed to simplify the process of building LLM-based applications.
Core building blocks	Model (unified interface to any LLM provider), Prompts (prompt engineering/templates), Retrievers (fetch relevant documents from a vector store/knowledge base), Chains (link components together).
Signature feature	Chains — connecting components so the output of one block automatically becomes the input of the next.
Typical use cases	Simple conversational workflows (chatbots, summarizers), multi-step linear workflows, RAG-based applications, basic tool-using agents.
Execution model	Sequential / linear — a chain runs start to finish without pausing.

Aspect	Description
Definition	An orchestration framework that enables building stateful, multi-step, and event-driven workflows using LLMs. Ideal for designing both single-agent and multi-agent applications.
Mental model	A flow-chart engine for LLMs. Each step is a node; nodes are connected using edges; edges carry the transition logic.
Built on top of	LangChain — LangGraph does not replace LangChain's components (models, prompts, retrievers, tools, document loaders, etc.). It adds an orchestration layer on top.
Representation	The entire workflow is represented as a graph (a non-linear data structure), so any level of complexity — branches, loops, jumps — can be modeled naturally.

6. When to Use What

Use LangChain when...	Use LangGraph when...
Building simple, linear workflows (prompt chains, summarizers, basic RAG systems).	Building complex, non-linear workflows.
No need for conditional branching or loops.	Conditional branches, loops, and jumps are required.
Short-running, synchronous tasks.	Long-running / event-driven execution is needed (pause and resume on external triggers).
No need to pause for human decisions.	Human-in-the-loop approval steps are required (possibly long-duration, asynchronous).
Single, self-contained flow.	Multi-agent coordination/collaboration or nested sub-workflows are needed.

7. Important Clarification: Don't Abandon LangChain

LangGraph is built on top of LangChain — it does not replace it. LangGraph's purpose is orchestration (chaining/coordinating a workflow), not providing the underlying components. Even in complex LangGraph-based systems, you still need LangChain's components:

- Chat model wrappers (e.g., ChatOpenAI)
- Prompt templates
- Retrievers
- Document loaders
- Text splitters
- Tools

In short: LangChain provides the components; LangGraph provides the orchestration layer for chaining and coordinating those components into complex, non-linear, stateful workflows. In real projects, both are typically used together — LangChain for the building blocks, LangGraph for the flow/control logic that ties them together.

8. Quick Revision Summary

1. A workflow is a fixed, developer-defined sequence of steps; an agent dynamically decides its own steps.
2. LangChain excels at simple, linear chains but struggles with non-linear control flow, state management, event-driven pauses, fault tolerance, human-in-the-loop waits, nested workflows, and full observability.
3. LangGraph solves all of the above by representing workflows as graphs with nodes, edges, and a shared, mutable state object, plus checkpointing for pausing/resuming/fault recovery.
4. Use LangChain for simple linear workflows (prompt chains, summarizers, basic RAG). Use LangGraph for complex, non-linear workflows needing conditionals, loops, human-in-the-loop, multi-agent coordination, or asynchronous event-driven execution.
5. LangGraph does not replace LangChain — it's built on top of it and used alongside it.